Datalaset:

Consider the following dataset:

Pattern(x_i)	Target (y_i)
(0.8, 0.4)	8
(0.4, 0.2)	8
(0.6, 0.4)	12
(0.8, 0.6)	16
(1.0, 0.7)	19
(0.8, 0.4)	14
(0.6, 0.2)	10
(0.5, 0.5)	12
(0.2, 0.6)	10

Predict target value $T = (0.3, 0.4)$ using $k=3$?

i) Compute equlidian distances b/w t & all traing points.

$d(T, x_i) = \sqrt{(0.3-0.2)^2 + (0.4-0.4)^2} = 0.100$

$= 0.224$

$= 0.300$

$= 0.539$

$= 0.762$

$= 0.500$

$= 0.361$

$= 0.224$

$= 0.224$

Comparison with linear regression

$y_i = a x_i(1) + b x_i(2) + c$

$y_1 = 8, (0.2, 0.4)$ $\quad 8 = a(0.2) + b(0.4) + c \rightarrow 1$

$y_2 = 8, (0.4, 0.2)$ $\quad 8 = a(0.4) + b(0.2) + c \rightarrow 2$

$y_3 = 12, (0.5, 0.5)$ $\quad 12 = a(0.5) + b(0.5) + c \rightarrow 3$

$a = 10,\ b = 10,\ c = 2$ 9.9.33

$\Rightarrow y = 10(0.3) + 10(0.4) + 2 = 9$

Error rate $= (9.33 - 9)^2 = 0.111$

Advantage:

→ Works well for small dataset with clear local patterns

Limitation:

→ choise of k affects the performance

Confusion Matrix

* purpose - find the modal's performance (accuracy, sensitivity).

TP	FN
FP	TN

TP → correct positive prediction.

TN → correct negative prediction.

FP → Incorrect positive predictions (or) Type-I error.

FN → Incorrect negative predictions (or) Type-II error

Example: Email spam/not.

Actual (y)	Predicted ($\hat{y}$)	
1	1	→ TP
0	0	→ TN
0	1	→ FP
1	0	→ FN

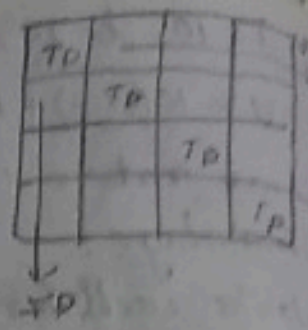
⇒ Let there be 3 classes C_1, C_2 and C_3

Let there be 200, 100 and 50 patterns from classes C_1, C_2 and C_3 respectively.

Let classifier classify the patterns into 3 classes

Time predicted	C_1	C_2	C_3
C_1	TP(180)	FN(15)	FN(5)
C_2	FN(5)	TP(85)	~~FP~~(10) FN
C_3	FN(3)	FN(2)	TP(45)

i) row sum is FN for 1st row
ii) column sum is FP for 1st column
iii) TN = Total − TP − FP − FN
iv) TP (diagonal) → all data points.



For class 2

• Confusion matrix : class 1

True	C_1	$\bar{C}_1$
C_1	TP 180	FN 20
$\bar{C}_1$	FP 8	TN 142

• Precision : It is the ratio of TP to the total number of predicted positives (TP+FP). 180/188 ≈ 0.9574
• Recall : It is the ratio of TP number of positives in the training data (TP+FN). 180/200 = 0.9
• F1 Score : mean of precision and recall;

$$\frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{recall}}} = 0.9278$$

⇒ Receiver operating characteristic (ROC)
• True positive rate or TPR = TP/TP+FN
• False positive rate or FPR = FP/FP+TN

⇒ Let the training patterns.
• class label 1 : (1,1), (2,2), (3,3), (4,4), (5,5)
• class label 0 : (4,1), (5,2), (6,3), (7,4), (8,5)
• Let the validation pattern be given by
• class label 1 : (2,1), (3,3)
• class label 0 : (8,4), (7,5)
• Let K = 5;

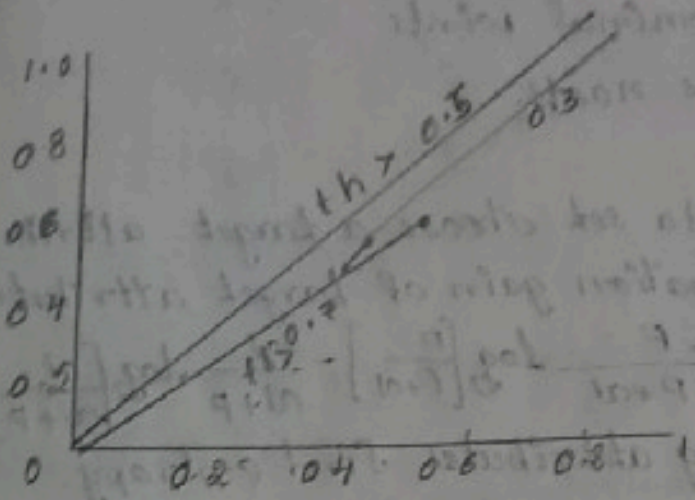
Pattern	class	5 NNs	Proba	th>0.3	th>0.5	th
(2,1)	1	(1,1)(2,2)(4,1)(3,3)(5,2)	0.6	1	1	0
(1,3)	1	(2,2)(1,1)(3,3)(4,4)(4,1)	0.8	1	1	1
(8,4)	0	(8,5)(7,4)(6,3)(5,5)(5,2)	0.2	0	0	0
(7,5)	0	(8,5)(7,4)(5,5)(6,3)(4,4)	0.4	1	0	0

• For Th > 0.3 we get TP = 2, FN = 0, FP = 0, TN = 2, so, FPR = 0/0+0 = 0 TPR = 2/(2+0) = 1

• For Th > 0.5 we get TP = 1, FN = 1, FP = 0, TN = 2, so, FPR = 0/2+0 = 0 & TPR = 1/(1+1) = 0.5

• For Th > 0.7 we get TP = 1, FN = 1, FP = 0, TN = 2, so, FPR = 0/2+0 = 0 & TPR = 1/(1+1) = 0.5

All are obtained by comparing class & threshold



th > 0.5 is best

Date: 11/09/2025

Note:

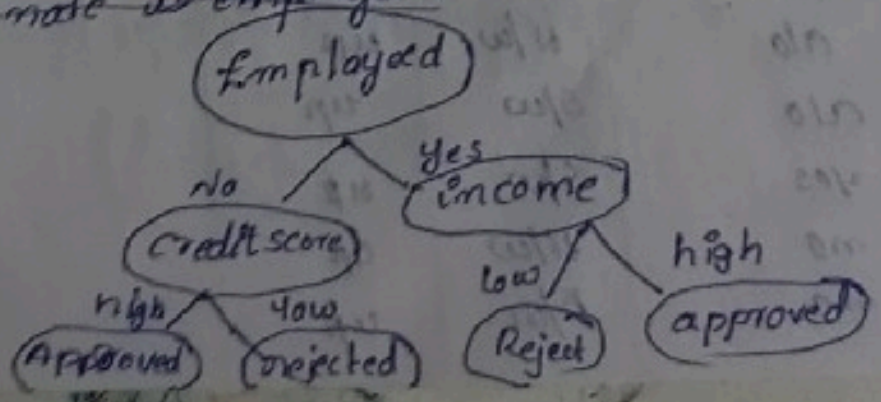
⇒ Metrics for classification algothim
Accuracy, Precision, recall, F1 score

⇒ Metrics for regression:
Mean absolute error, mean squared error (or) Squared error.

~~⇒ Decision tr~~

Decision tree
Which is used in tree structured classification & Regression

Eg: loan system
. Decides if the loan should approved (or) rejected.



Root node: This is the starting point representing entire data set.

Splitting: The algorithm selects the best feature and threshold to split the data based on a criteria.

Example: gini index (or) Information gain.

Decision node: Intermediate point where the dataset is further divided.

Leaf nodes: Terminal points

Note: edges means roots.

ID3 Algorithm

1) In the given data set choose a target attribute

2) Calculate information gain of target attribute

$$IG = -\frac{P}{P+N}\log_2\left[\frac{P}{P+N}\right] - \frac{N}{N+P}\log_2\left[\frac{N}{N+P}\right].$$

3) for remaining attributes find entropy.

$$\text{entropy} = IG * \text{probability}.$$

Entropy of every attribute $E(A) = \sum \frac{P_i+N_i}{P+N} * IG(P_i,N_i)$

4) calculate gain

$$\text{gain} = IG - E(A).$$

Example: Dataset (apply table encoding for program)

Age	competetion	type	profit
old	yes	s/w	Down
old	no	s/w	Down
old	no	Hardware	Down
mid	yes	s/w	Down
mid	yes	h/w	Down
mid	no	H/w	up
mid	no	s/w	up
new	yes	s/w	up
new	no	H/w	up
new	no	s/w	up

i) target attribute is profit

ii) Calculate IG of profit

$$IG = \frac{-P}{P+N}\left\{\log_2\left[\frac{P}{P+N}\right]\right. - \frac{N}{N+P}\log_2\left[\frac{N}{N+P}\right]$$

P = Count(down) = 5

N = count (up) = 5

$$IG = \frac{-5}{10}\log_2\left[\frac{5}{10}\right] - \frac{5}{10}\log_2\left[\frac{5}{10}\right]$$

$$= -\frac{1}{2}\log_2\left[\frac{1}{2}\right] - \frac{1}{2}\log\left[\frac{1}{2}\right]$$

$$= -\frac{1}{2}\log_2 2^{-1} - \frac{1}{2}\log_2 2^{-1}$$

$$= -\frac{1}{2}(-1)\log_2 2 - \frac{1}{2}(-1)\log_2 2$$

$$= 1.$$

iii) Calculate entropy for remaing attributes (age, competition, type).

now, prepare table for each attribute.

rows → (old, mid, new)

column (values of target attributes) ⇒ (down, up)

	down	up
old	3 (P)	0 (N)
mid	2	2
new	0	3

$$\Rightarrow IG(old) = -\frac{3}{3}\log_2\left[\frac{3}{3}\right] - \frac{0}{3}\log_2\frac{0}{3}$$

$$= -1\log_2 1 = 0. \qquad [\because \log_2 1 = 0]$$

Probability of old $= \frac{3}{10} =$

Entropy of old = IG × probability

$$= 0 \times \frac{3}{10}$$

$$= 0$$

$\Rightarrow IG(mid) = -\left(\frac{2}{4}\log_2\frac{2}{4} - \frac{4}{4}\log_2\frac{2}{4}\right)$

$= -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2}$

$= \qquad = 1.$

Probability of mid $= \frac{4}{10} = 0.4$

Entropy of mid $= IG * prob = 1 \times 0.4 = 0.4$

$\Rightarrow IG(new) = -\left(\frac{0}{3}\log_2\frac{0}{3} + \frac{3}{3}\log_2\left(\frac{3}{3}\right)\right)$

$=$

$= 0$

Probability of mid $= \frac{3}{10} = 0.3$

Entropy(new) $= 0 * 0.3 = 0.$

$*$ Entropy (age) $= E(old) + E(mid) + E(new)$

$= 0 + 0.4 + 0 \cancel{= 0.4}$

$= 0.4$

Gain $= IG -$ Entropy (Age)

$= 1 - 0.4 = 0.6.$

In the same way calculate gain of other attributes

	down	up
Yes	3	1
No	2	4

for (competition)

$\Rightarrow IG(competition) = -\left(\frac{3}{3+1}\log_2\frac{3}{3+1} + \frac{1}{3+1}\log_2\frac{1}{3+1}\right)$

$= -\left(\frac{3}{4}\log_2\frac{3}{4} + \frac{1}{4}\log_2\frac{1}{4}\right]$

$= -3\left[\frac{3}{4}\log_2 3^{-1} + \frac{1}{4}\log_2 4^{-1}\right]$

$=$

Probability of (yes) = $\frac{4}{10} = 0.4$

Entropy of yes = IG & prob = $\times 0.4 =$

$\Rightarrow IG(NO) = -\left[\frac{2}{2+4} \log_2 \frac{2}{2+4} + \frac{4}{2+4} \log_2 \frac{4}{2+4}\right]$

$= -\left[\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3}\right]$

$= -[$

Probability of (NO) = $\frac{6}{10} = 0.6$

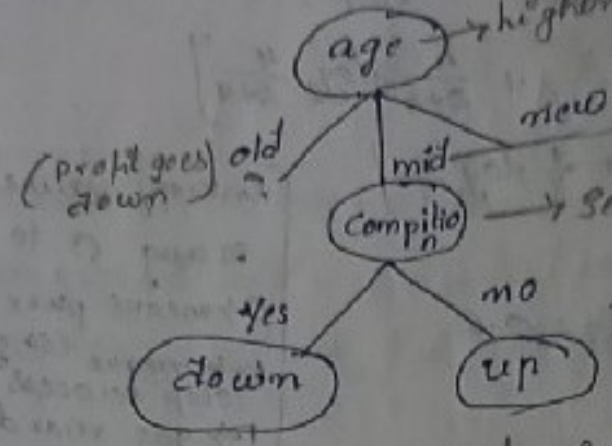
Entropy

Entropy value
rang 0 to 1
0 → means pure set
~~1 → means~~ (so you can stop process
1 → gos max disorder

$-\left(\frac{1}{3} \times -1.58 + \frac{2}{3} \times 0\right.$

$-(-0.52 + -0.38$

→ Highest gain will be as the root node.



why not other attribute in next step means gain is zero and decision tree also not require

Challenges :-

i) Greedy approach (Selecting best feature for current split

ii) Competitionally expensive. (depends on the no of features and data points.

iii) prone through overfitting (when accuracy is 100%)

sol → when overfitting occur
↳ remove unneccesary braches based on performance on a validation ~~set~~ test set.

How to address these challenges.

i) Random forest algorithm (ensembling) → selects all but determines which has the major votings.

ii) Ada boost algorithm

iii) Gradient Descent : means finding error rate if error rate is high we need to reduce weight. (By multiplying 0.01, 0.001 and update weights. ↳ learning rate.

Decision tree classification Using Gini Index

age	competition	type	profit
old	Yes	S/w	Down
old	No	S/w	Down
old	No	H/w	Down
mid	Yes	S/w	Down
mid	Yes	H/w	Down
mid	No	H/w	up
mid	No	S/w	up
new	Yes	S/w	up
new	no	H/w	up
new	no	S/w	up.

Step 1: Identify the target attribute and compute Gini impurity.

Here target attribute : profit

Gini Impurity : $1-\sum_{i=1}^{n} P_i^2$ ← where P_i is probability of all classes.

Profit : up(5)

Down(5)

$P(down) = \frac{5}{10} = 0.5$; $P(up) = \frac{5}{10} = 0.5$.

Now calculate Gini Impurity.

Gini Impurity : $1-(0.5^2+0.5^2)$

$= 1-(0.25+0.25) = 1-0.5$

$= 0.5$

Step2: Evaluate each attribute gini impurity.

Age attribute

old : 3(all down) | ~~mid~~ $P(down) = \frac{3}{3} = 1$ $P(up) = \frac{0}{3} = 0$

∴ Gini of old : $1-(1^2+0^2) = 0$.

~~mid att~~ Mid

mid: 2(down) 2(up) | $P(down) = \frac{2}{4} = 0.5$ | $P(up) = \frac{2}{4} = 0.5$

Gini (mid) = $1-(0.5^2+0.5^2)$
$= 0.5$

new

Gini(up) | $P(up) = \frac{3}{3} = 1$ | $P(down) = \frac{0}{3} = 0$

Gini(new) : $1-[1^2+0^2] = 0$.

~~Step~~ : Weighted gini impurity (Age) = ~~Probab~~
= probability(old) * gini(old) + P(mid) * gini(mid) + P(new) * Gini(new)

$= \frac{3}{10} * 0 + \frac{4}{10} * 0.5 + \frac{3}{10} * 0$

$= 0.2$

Gini Gain(Age) = $0.5 - 0.2 = 0.3$.

Competition Attribute:-

Categories (yes no)

yes : 3(down), 1(up) | $P(down) = \frac{3}{4}$ 0.75 | $P(up) = \frac{1}{4} = 0.25$

Gini(yes) = $1-(0.75^2+0.25^2)$
$= 0.375$

no : 4(up), 2(down) | $P(down) = \frac{2}{6} = 0.33$ | $P(up) = \frac{4}{6} = 0.667$

Gini(no) = $1-(0.33^2+0.667^2)$
$= 0.445$.

Weighted gini Impurity
= P(yes) * Gini(yes) + P(no) * gini(no)

$= \frac{4}{10} * 0.327 + \frac{6}{10} * 0.445 = 0.417$

Gini gain of competition = $0.5 - 0.447$
$= 0.083$

Type attribut

Categories (S/w, HW)

S/w : 3 (2 down) 3(up) | $P(down) = \frac{3}{6} = 0.5$ | $P(up) = \frac{3}{6} = 0.5$

Gini(S/W) = ~~1 - (0.4² + 0.6²) = 0.48~~ 1 - (0.5² + 0.5²) = 0.5

H/W : (2 down, 2 up) = | P(down) = $\frac{2}{4}$ = 0.5 | P(up) = $\frac{2}{4}$ = 0.5)

Gini(H/W) = 1 - (0.5² + 0.5²) = 0.5

Weighted gini impurity

= P(S/W) * gini(S/W) + P(H/W) * gini(H/W)

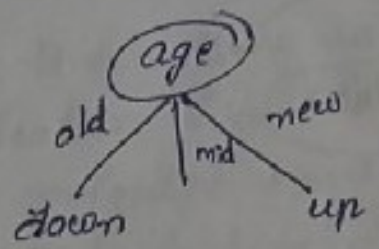
= $\frac{6}{10}$ * 0.5 + $\frac{4}{10}$ * 0.5

= 0.3 + 0.2 = 0.5

Gini Gain(type) = 0.5 - 0.5 = 0

Step 3: choose Root node. age has the highest gini gain so it becomes the root node.

Step 4: Built the tree.



mid contains ~~multi~~ mixed notes needs further splitting

age	Compe	Type	Profit
mid	yes	s/w	down
mid	yes	H/w	down
mid	no	H/w	up
mid	no	s/w	up

~~Competition attribute:~~

~~categories (yes, no)~~ Gini(profit)

P(up) = $\frac{2}{4}$ = 0.5 P(down) = $\frac{2}{4}$ = 0.5

Gini(down) = 1 - (0.5)² + 0.5²) = 0.5

Competition attribute : 4 (2 up, 2 down) (yes, no)

P = $\frac{2}{4}$ = 0.5 ,, P(up) = $\frac{2}{4}$ = 0.5

Gini(yes) = ~~1 - (0.5² + 0.5²) = 1~~ , 2 (down) Gini(yes) = 1 - (1² + 0²) = 0

Gini(No) = 2 (all up) | Gini(No) = 1 - (0² + 1²) = 0.

Gini Impurity of competition $= \frac{2}{4} \times 10 + \frac{2}{4} \times 10 = 0$.

Gini gain of competition $= 0.5 - 0 = 0.5$ → perfect split

Type for mid

s/w = (1 up, 1 down)

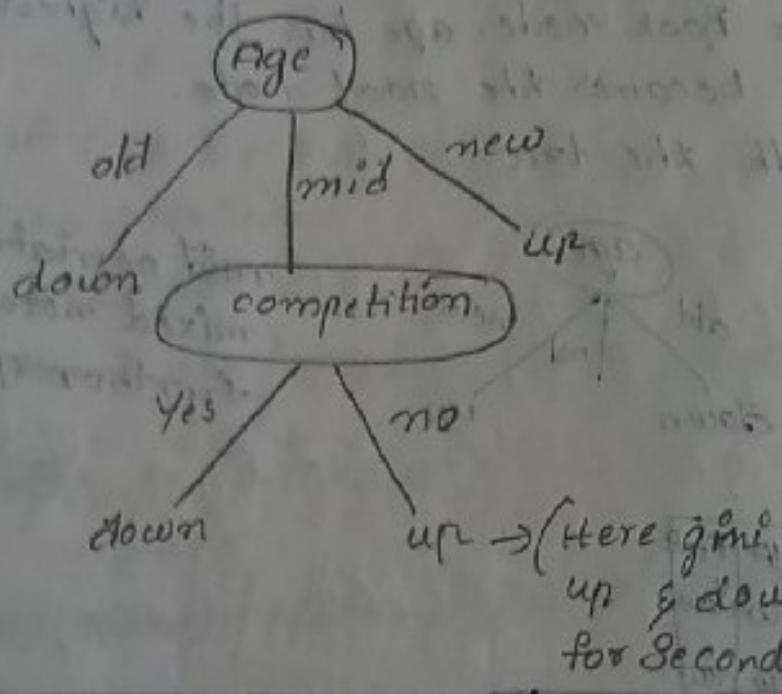
$Gini(s/w) = 1 - (0.5^2 + 0.5^2) = 0.5$

H/w : 2 (1 up, 1 down)

$Gin(H/w) = 1 - (0.5^2 + 0.5^2) = 0.5$

Impurity of Gini $= \frac{1}{2} \times 0.5 + \frac{1}{2} \times 0.5 = 0.5$

Gini Gain of type $= 0.5 - 0.5 = 0$.



Note: If we are using numerical values tree will be consider based on threshold.

1) For categorical no threshold value.

For foundational of theorical root calculate entropy (0 to 1) $\left[-\sum P_i \log_2 P_i\right]$ → Shenon entropy.

$\log_2$ → Here base 2 for binary cos)

$$-\left[\frac{P}{P+N} \log_2 \frac{P}{P+N} + \frac{N}{N+P} \log \frac{N}{N+P}\right]$$ } also for entropy.

2) Gini Index (0 - 0.5) range.

It is computationally efficient

Gini Index : $1 - \sum_{i=1}^{n} P_i^2$

3) Misclassification value range 0 to 0.5

formula = $1 - \text{Max}(P_i)$

It is used to simplest compute and understand.

To identify best feature selection algorithm

Overview

1) Begin with the entire dataset at the root node.
2) For each feature find the split point that minimizes the weighted squared error. (i.e, searching)
3) Choose the best feature and split point from step 2. partition the data into two child node
4) Recursively repeat steps 2 to 3 for each new child node.
5) Continue until a stoping condition met (node purity / minimum size) declare a node as leaf and assign it the mean 'y' values of its training samples.
6) For a new sample navigate the tree from the root to a leaf based on its feature values the prediction is the mean valued stored in that leaf.

* Bias Variance Trade off

↳ Training (error rate occurs) that is Bias

↳ Testing (error rate occurs) that is variance

↳ High bias means → under fitting
High variance means → over fitting

Trade off :-
Cannot simultaneously minimise variance and bias

If reduces variance, then increases bias
If reduces bias, then increases variance
Goal: is the reduce the error. [Trade off]

Model1:
The simple linear model (underfitting/high bias).
Model type is: $y = mx + c$.
for the three training points (0,1), (1,3), (2,7).

for point (0,1) ⇒ $1 = (0)m + c$ ⇒ $c = 1$.
for point (1,3) ⇒ $3 = m(1) + c$ ⇒ $m + c = 3$
for point (2,7) ⇒ $7 = 2m + c$ ⇒ $2m + c = 7$.

Δ_1^2 To find m & c

Least square method to find best possible m & c for our traing data :-

$\Sigma y = c * n + m * \Sigma x$ → (1)
$\Sigma xy = c * \Sigma x + m * \Sigma x^2$ → (2)
Here n = no. of data points.
x, y are the data points.

Let $x = 3$.

$\Sigma x = 0 + 1 + 2 = 3$
$\Sigma y = 1 + 3 + 7 = 11$
$\Sigma xy = (0 \times 1) + (1 \times 3) + (2 \times 7) = 17$
$\Sigma x^2 = 0^2 + 1^2 + 2^2 = 5$

from equation (1)
⇒ $11 = c * 3 + m * 3$

from equation (2)
⇒ $17 = c * 3 + m * 5$

$3c + 3m = 11$
$3c + 5m = 17$
$m = 3 \quad c = 2/3$.

$$\boxed{\therefore y = 3x + \frac{2}{3}}$$

Now evaluating model performance on the training data (This shows bias)

for $x=0$; $y = 3*0 + \frac{2}{3} = 0.66$ // Error $= (1-0.66) = 0.34$

$x = 1$; $y = 3*1 + \frac{2}{3} = 3.66$ // Error $= (3-3.66) = -0.66$

$x = 2$; $y = 3*2 + \frac{2}{3} = 6.66$ // Error $= (7-6.66) = 0.34$

The model makes mistakes even on the data it was trained on. So this is high bias or underfitting (The model is too simple to capture real patterns)

On test data (This shows variance), T (0.5, 1.75)

So $x = 0.5$; $y = 3*0.5 + \frac{2}{3} = 2.166$ / error $= (1.75 - 2.166) = -0.416$

It also performs poorly on new data

∴ Model 1 example for underfitting.

Model 2 :

The High variance model

→ for complex polynomials (or) overfitting

model equation : $y = ax^3 + bx^2 + cx + d$.

Now the data points are → (0,1), (1,3), (2,7)

for (0,1) → $1 = a(0)^3 + b(0)^2 + c(0) + d$

(1,3) → $3 = a(1)^3 + b(1)^2 + c(1) + d \Rightarrow a + b + c = 2 \rightarrow (1)$

(2,7) → $7 = a(2)^3 + b(2)^2 + c(2) + d \Rightarrow 8a + 4b + 2c = 6 \rightarrow (2)$

Solve equation (1) & (2) ∴ a can be any number (or) free variable

$b = 1 - 3a$

$c = 1 + 2a$; $d = 1$

then $\boxed{y = ax^3 + (1-3a)x^2 + (1+2a)x + 1}$

If $a = 1/4$; $b = 1/4$; $c = 2/4$; $d = 1$

then $\boxed{y_1 = \frac{1}{4}x^3 + \frac{1}{4}x^2 + \frac{3}{4}x + 1}$

If $a = 1/3$; $b = 0$; $c = 5/3$; $d = 1$

then $\boxed{y_2 = \frac{1}{3}x^3 + 0x^2 + \frac{5}{3}x + 1}$

↳ Evaluate model performance

On training data $x=0,1,2$ $y=1,3,7$

(y_1, y_2 satisfy the equation for 3 traing points

$\therefore$ Error $=0$.

So this means low bias.

The model is complex enough to fit the training data exactly.

On test data $x=0.5$, $y=1.75$, This is the problem of high variance

for y_1 equation :

$$y_1 = 1 + \frac{3}{2} * 0.5 + \frac{1}{4} \times (0.5)^2 + \frac{1}{4} * (0.5)^3$$

$$y_1 = 1 + (1.5)(0.5) + (0.25)(0.25) + (0.25)(0.125)$$

$$\boxed{y_1 = 1.84375}$$

Similarly for $y_2 = 1 + \frac{5}{3} * (0.5) + \frac{1}{3}(0.5)^3$

$$= 1 + (1.66)(0.55) + (0.33)(0.125)$$

$$\boxed{y_2 = 1.875}$$

Both models which are equally perfect on the training data but they give different & incorrect predictions for the test point.

This inconsistance is High variance.

Model 3 :- Balanced Model

Model Equation : $y = ax^2 + bx + c$ $\therefore \underbrace{(0,1)\ (1,3)\ (2,7)}_{\text{Actual values}}$

So $(0,1) \Rightarrow 1 = a(0)^2 + b(0) + c$

$(1,3) \Rightarrow 3 = a(1)^2 + b(1) + c$

$(2,7) \Rightarrow 7 = a(2)^2 + b(2) + c$

So after solving $a=1, b=1, c=1$

So the model is $y = x^2 + x + c$.

Model performance

On training data perfect fit - (low bias] T $(0.5, 1.7$

$$y = (0.5)^2 + (0.5) + 1 = 0.25 + 0.5 + 1 = 1.75$$

Model type	$y=mx+c$	$y=ax^2+bx+c$	$y=ax^3+bx^2+cx+d$
Bias	High	low	low
Variance	low	~~High~~ low	High
	↓	↓	↓
	under fitting	good generalization	over fitting

25/sep/2025

Decision Tree classifier Properties:

1) Splitting rules:
   i) Axis parallel splits
      Ex: $f_1 < 0.4$, $f_2 < 0.8$ etc
   ii) oblique splits
      Ex: [constant * feature], $20*f_1 + 40*f_2 > 100$.

2) Splitting criteria:
   → Through Information Gain
   → Through Gini Index

3) Tree Structure (Binary / non-binary)
   ↓ if 2 child for node
   ↳ Ex: yes or no
   ↳ if multiple child for node.
   ↳ Ex: for categorical features.

4) Termination Conditions.
   → Stop when a node is pure
   → Stop when the impurity level is below a small threshold (1-2%)
   → Use a validation set to stop when performance no longer improves.

5) Class lables for leaf nodes.
   If a leaf node is pure it is labeled with that single class
   If a leaf node is impure it is labelled with majority class of the pattern with node

6) Classification of new patterns:
→ Test pattern is rooted from down the tree from root to a leaf node based on the test conditions at each node.

7) Transpency
→ Each path from the root to a leaf clear if-then rule making predictions.

8) Handling Mixed types.
→ Both numerical & categorical features

9) Eliminating Irrelevent features.
→ Tree construction process automatically selects most relevant features.

10) Pruning.
→ The prevent is overfitting.
overfitting occur when working with largedata
→ Pre-pruning :- stop growing the tree early based on (maximum depth or minimum number of Patterns) is reached.
→ post-pruning : let the tree grow fully then remove branches that seem to model noise (nodes with very few patterns)

(12) Decision tree as a Regressor

```
import numpy as np
from sklearn.tree import DecisionTree Regressor
import matplotlib.pyplot as plt
#(12) Dataset
X = np.array([1500, 1700, 2500, 2700, 3000]).reshape(-1,1)
y = np.array([200000, 220000, 350000, 370000, 400000])
# Create & train the decision tree regressor
regressor = DecisionTree Regressor(max_depth = 3)
```

```
regressor.fit(X,y)
# predic house prices
X_test = np.linspace(1400, 3100, 200).reshape(-1,1)
```

↳ $(3100-1400)/(200-1) = 5.68$

1400 1405.68 3100

y_1 y_2

```
y_predict = regressor.predict(X_test)
#plot the decision tree predictions
plt.scatter(X, y, color='blue', label='Actual data')
plt.plot(X_test, y_predict, color='red', label='predicted
        curve')
plt.title('Decision Tree Regression')
plt.xlabel("Square feet area")
plt.ylabel("House price")
plt.legend() → shows actual predicted lables
```

Random forest:

A random forest is a powerfull machine learning algorithms that builds a collection of decision trees during training time for both classification and regression tasks.

It makes prediction by aggregating the outputs of multiple trees using

⇒ Boot strapping (Bagging technique)

- Random forest uses bootstrap sampling where random subsets of data points are selected with replacemen to train each decision tree.
- This ensures that trees are trained on slightly different data sets making the model robust & less prone to overfitting

convergence
↳ Both previous & present error and accuracy same we stop process

⇒ Feature Randomness

- At each split with in a tree only a random

random subset of features to find best split instead of all the features.

- This tree correlates that tree and reduces the tree likelyhood of all trees converging to the same

Solution → probability (error or accuracy) with random state - same error or accuracy for same seeds

⇒ Aggregation

- For classification the final prediction is made based on majority voting (The most common class across all trees)
- For Regression the final output is the average of all the individual tree predictions

Why use Random forest?

i) Improved Accuracy :- Multiple trees reduce errors and variance.

ii) Resilience to Overfitting: Bootstrapping and random feature selection prevent overfitting.

100 → 10 10 - - -

iii) Robust to Noise and Outliers: Reduces the impact of noisy data points.

iv) Feature Importance: Naturally ranks feature important during Training.

Program

```
import numpy as np
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt
#training data:
X = np.array([1500,3], [1700,4], [2500,4], [2700,5]
             [3000,4])
y = np.array([200000, 250000, 350000, ...])
```

#Now create & train random forest model
regressor = RandomForestRegressor(n_estimators = 100,
random_state = 42)

with random state predictors are identical and model performance stabe

with random state feature importance changes each run

#test data prediction
X_test = np.array([1600, 3], [2800, 5])
predictions = regressor.predict(X_test)
print(predictions) → user defined.

#visualizing feature importance
importances = regressor.feature_importances_ (shows which feature most influences price prediction (area, no. of rooms))
feature_names = ['area' and 'rooms']
plt.barh(feature_names, importances)
plt.title('Feature Importance')
plt.show()

Features & Advantages:

- It is one of the most accurate learning algorithm.
- for many dataset it produces a highly accurate classifier.

2) It runs efficiently on large databases.
3) It can handle thousands of input variables without variable deletion.
4) It gives estimates of what variables are important in classification.
5) It generates an internal unbiased estimate of the generalization error as the forest building progresses.

c) It has an effective method for estimating missing data and maintains accuracy when a large proportion of the data are missing.

01/10/2024